👌 XAI Lecture 06

Generalized Additive Models & Prototype-Based Approaches

FAMAF - UNC

First Semester 2026

Disclaimer ---

This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

https://interpretable-ml-class.github.io/

Intelligible Models for HealthCare: Predicting Pneumonia Risk and Hospital 30-day Readmission

Rich Caruana
Microsoft Research
rcaruana@microsoft.com

Yin Lou
LinkedIn Corporation
ylou@linkedin.com

Johannes Gehrke
Microsoft
johannes@microsoft.com

Paul Koch
Microsoft Research
paulkoch@microsoft.com

Marc Sturm
NewYork-Presbyterian Hospital
mas9161@nyp.org

Noémie Elhadad
Columbia University
noemie.elhadad@columbia.edu

ABSTRACT

In machine learning often a tradeoff must be made between accuracy and intelligibility. More accurate models such as boosted trees, random forests, and neural nets usually are not intelligible, but more intelligible models such as logistic regression, naive-Bayes, and single decision trees often have significantly worse accuracy. This tradeoff sometimes limits the accuracy of models that can be applied in mission-critical

the application of machine learning to important problems in healthcare such as predicting pneumonia risk. In the study, the goal was to predict the probability of death (POD) for patients with pneumonia so that high-risk patients could be admitted to the hospital while low-risk patients were treated as outpatients. In the study [3, 2], the most accurate models that could be trained were multitask neural nets.[1] On one dataset the neural nets outperformed traditional methods such as logistic regression by wide margin (the neural

- Generalized Additive Models -

(Caruana et al. 2015)

Contributions ---

Two healthcare case studies are presented: - **Pneumonia risk prediction** — small dataset (14K patients, 46 features) - **30-day hospital readmission** — large dataset (196K patients, 3,956 features)

Main Claim

GA^{2}Ms achieve **state-of-the-art accuracy** while remaining **intelligible, modular, and editable** — challenging the assumption that accuracy and interpretability must be traded off..

Roadmap

1. Motivation
2. Intelligible Models
3. Case study: Pneumonia risk
4. Case study: 30-day readmission

Motivation ---

- A large project to evaluate application of ML to healthcare problems
- Predicting **probability of death (POD)** for pneumonia patients
  - **Most accurate model:** Neural nets (0.86 AUC)
  - **Actually Deployed:** Logistic Regression: (0.77 AUC)

Why was logistic regression used instead?

Motivation: The Asthma Paradox ---

- 1990s study: neural nets were the **most accurate** models for pneumonia mortality
- A rule-based system discovered: `HasAsthma(x)` $\Rightarrow$ `LowerRisk(x)`
  - Asthmatic patients were sent directly to the ICU $\rightarrow$ aggressive care $\rightarrow$ lower observed mortality
  - The model learned the **effect of treatment**, not the true underlying risk

The Core Problem: The NN almost certainly learned the same pattern

But because it was **opaque**, there was no way to detect or fix it. Deploying it could have sent high-risk asthmatic patients home.

The Lesson

- A dangerous rule in an intelligible model can be **recognized and removed**.
- A dangerous pattern in a black-box model may never be found.

SVMs, random forests, boosted trees, deep nets — all unintelligible

Why GA²Ms

Accurate as black-box models — but **intelligible**, **modular**, and **editable** by domain experts.

Generalized Additive Models (GAMs) ---

Intuition: Instead of one global equation, the model learns a separate function for each feature — then adds them up.

$$g(\mathbb{E}[y]) = \beta_0 + \sum_j f_j(x_j)$$

where g is a link function and each f_j is a **shape function** learned from data, with $\mathbb{E}[f_j] = 0$.

Key Properties

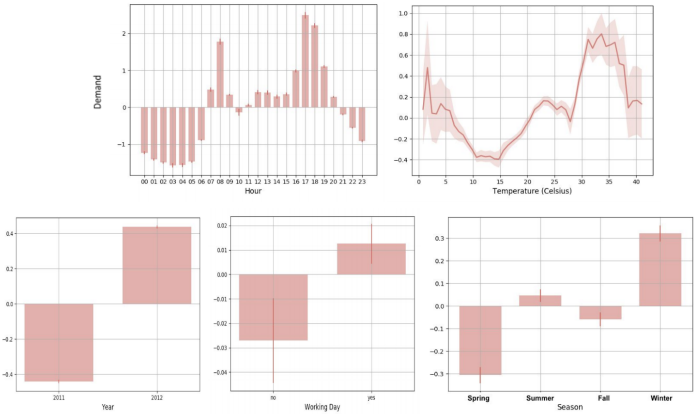
- Each f_j can be **non-linear** — no need to manually discretize features
- Contributions are **additive and independent** — each f_j can be visualized separately
- Logistic regression is a special case where $f_j(x_j) = w_j x_j$

GAMs: Notation ---

$$g(\mathbb{E}[y]) = \beta_0 + \sum_j f_j(x_j)$$

- g — **link function**: maps the expected output to the linear predictor. For binary classification: $g(p) = \log \frac{p}{1-p}$ (logit)
- f_j — **shape function**: learned from data, centered so that $\mathbb{E}[f_j] = 0$, ensuring each term has a unique, identifiable contribution.
  - $\mathbb{E}[f_j] = 0$ The average of the shape function over the training data is zero:
- β_0 — **baseline**: the only constant term; calibrated so that the average predicted probability equals the observed baseline rate in the training data

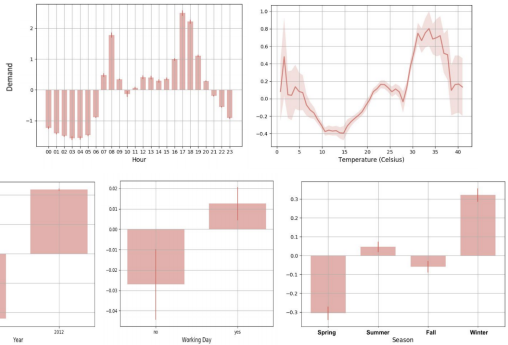
GAMs -- Shape Functions (Bike Sharing) ---



Shape functions learned by a GAM on a bike-sharing dataset.

Each plot shows the contribution of one feature to predicted demand (y-axis: additive score relative to baseline)

GAMs -- Shape Functions (Bike Sharing) ---



Features shown:

- **Hour** (top-left) — demand peaks at 8am and 5-6pm, reflecting commute patterns;
- **Temperature** (top-right) — demand rises with warmth, drops above ~35°C;
- **Year** (bottom-left) — demand grew from 2011 to 2012;
- **Working Day** (bottom-center) — slight increase on working days;
- **Season** (bottom-right) — winter highest, spring lowest.

$$g(\mathbb{E}[y]) = \beta_0 + f_1(x_1) + f_2(x_2) + \cdots + f_p(x_p)$$

Each f_j is learned freely from data — no linearity assumption required.

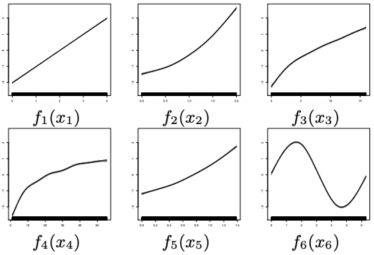


Figure 1: Shape functions can take any form: linear, concave, oscillating.

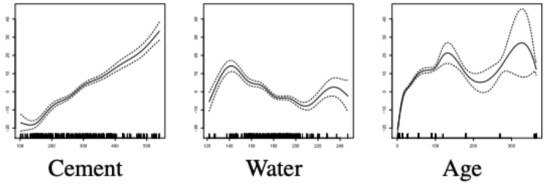


Figure 2: **Cement:** monotonically increasing — more cement, higher strength. **Water:** non-monotone — there is an optimal range; too much or too little reduces strength. **Age:** rapid early growth (curing process), stabilizes with a second soft peak at later ages.

From GAM to GA^{2}M ---

GAMS

$$g(\mathbb{E}[y]) = \beta_0 + \sum_j f_j(x_j)$$

A GAM models each feature independently — the contribution of x_j to the output never depends on the value of any other feature.

GA^{2}M

$$g(\mathbb{E}[y]) = \beta_0 + \sum_j f_j(x_j) + \sum_{i \neq j} f_{ij}(x_i, x_j)$$

Adds pairwise interaction terms $f_{ij}(x_i, x_j)$, capturing cases where the effect of one feature depends on another. - First fits the best GAM, then detects and ranks all pairwise interactions in the residuals. - The top k pairs are added.

Intelligibility and Accuracy ---

Model	Form	Intelligibility	Accuracy
Linear Model	$y = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$	+++	+
Generalized Linear Model	$g(y) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$	+++	+
Additive Model	$y = f_1(x_1) + \ldots + f_n(x_n)$	++	++
Generalized Additive Model	$g(y) = f_1(x_1) + \ldots + f_n(x_n)$	++	++
Full Complexity Model	$y = f(x_1, \ldots, x_n)$	+	+++

(Lou, Caruana, and Gehrke 2012)

Learning Shape Functions ---

Shape functions f_j can be represented as **splines** or **regression trees**. The paper uses gradient boosting with bagging of shallow trees — empirically the most accurate choice.

Training GA^{2}M

1. Fit the best GAM using gradient boosting on individual features
2. Compute residuals and detect all possible pairwise interactions
3. Rank interactions by their ability to explain the residuals
4. Add the top k pairs to the model (k chosen by cross-validation)

Bagging (100 rounds for pneumonia) reduces overfitting and provides pseudo-confidence intervals for the shape plots.

Case Study: Pneumonia Risk ---

Dataset

- 14,199 pneumonia patients
- Train: 9,847
- Test: 4,352
- 46 features

Task

Predict **probability of death (POD)**

10.86% of patients died (1,542)

Pneumonia Risk: Features ---

Patient-history findings			
chronic lung disease	-	age	C
re-admission to hospital	-	gender	-
admitted through ER	-	diabetes mellitus	-
admitted from nursing home	-	asthma	-
congestive heart failure	-	cancer	-
ischemic heart disease	-	number of diseases	C
cerebrovascular disease	-	history of seizures	-
chronic liver disease	-	renal failure	-
history of chest pain	-		
Physical examination findings			
diastolic blood pressure	C	wheezing	-
gastrointestinal bleeding	-	stridor	-
respiration rate	C	heart murmur	-
altered mental status	-	temperature	C
heart rate	C		

Laboratory findings			
liver function tests	-	BUN level	C
glucose level	C	creatinine level	C
potassium level	C	albumin level	C
hematocrit	C	WBC count	C
percentage bands	C	pH	C
pO2	C	pCO2	C
sodium level	C		
Chest X-ray findings			
positive chest x-ray	-	lung infiltrate	-
pleural effusion	-	pneumothorax	-
cavitation/empyema	-	chest mass	-
lobe or lung collapse	-		

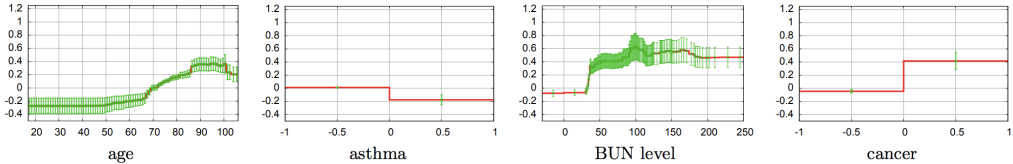
- **C (continuous):** numeric feature — the GAM learns a full shape function $f_j(x_j)$ that can be non-linear
- **— (binary/categorical):** takes values $0/1$ — the shape function reduces to two points; presented as a bar plot for visual consistency with continuous features

Pneumonia Risk: AUC Results ---

Model	Pneumonia (AUC)
Logistic Regression	0.8432
GAM	0.8542
GA^{2}M	0.8576
Random Forests	0.8460
LogitBoost	0.8493

- GA^{2}M achieves the highest AUC while remaining fully intelligible.
- The gap between models is small (<0.02).
- **Note** that the dataset is highly imbalanced (only **10.86%** of patients died), which can bias AUC as an evaluation metric.

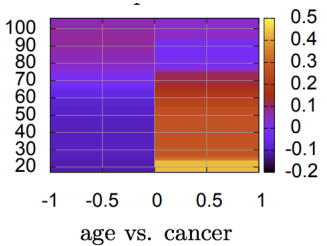
Understanding Outputs of GA^{2}M (1/2) ---



Shape functions learned by GA^{2}M on the pneumonia dataset (y-axis: risk score contribution relative to baseline).

- **Age:** low and flat below 50, rises sharply after 65.
- **Asthma:** having asthma *decreases* predicted risk — a known data artifact (ICU effect).
- **BUN (Blood Urea Nitrogen) level:** missing/low values indicate low risk; elevated levels ($>$30) signal kidney or cardiac complications.
- **Cancer:** strong positive risk contribution — nearly doubles the predicted odds of death.

Understanding Outputs of GA^{2}M (2/2) ---



The interaction term $f_{ij}(\text{age}, \text{cancer})$ reveals a pattern invisible to individual shape functions:

- **Cancer = 1, young patients:** highest risk (yellow) — childhood cancers are associated with high risk of death
- **Cancer = 1, older patients:** moderate risk (orange/red) — adult cancers are serious but less acutely lethal
- **Cancer = 0:** uniformly low risk (purple) regardless of age

Case Study: 30-Day Readmission ---

Dataset

- 195K patients (train)
- 100K patients (test)
- **3,956 features**

Context

Hospitals with high readmission rates are **penalized financially** (inadequate earlier care).

8.91% of patients readmitted within 30 days.

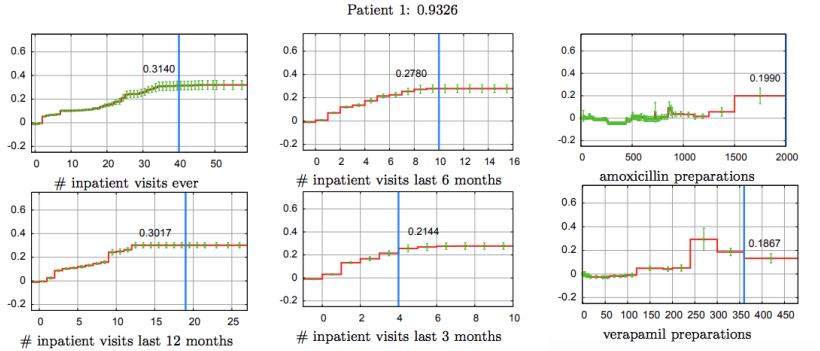
30-Day Readmission: AUC Results ---

Model	Readmission (AUC)
Logistic Regression	0.7523
GAM	0.7795
GA^{2}M	0.7833
Random Forests	0.7671
LogitBoost	0.7835

- GA^{2}M matches LogitBoost (0.7833 vs 0.7835) while remaining fully intelligible — on a dataset with 196K patients and 3,956 features.
- Again, the dataset is imbalanced (**8.91%** readmission rate), so AUC should be interpreted with caution as it may overestimate model performance.

Case Study: High Risk Patient ---

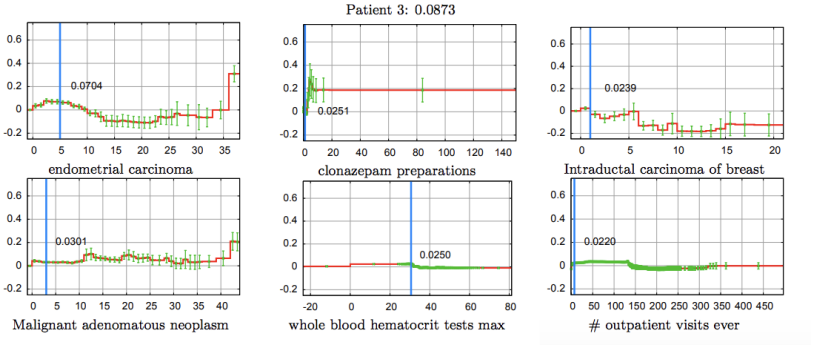
Top 6 shape functions sorted by risk contribution for a High Risk Patient ($p = 0.9326$)



- The blue line marks the patient's feature value; the number is its risk score contribution.
- High readmission risk is driven by a history of frequent hospitalizations (40 total, 19 in the last 12 months)
- and large doses of amoxicillin — suggesting an ongoing infection not responding to antibiotics.

Case Study: Low Risk Patient ---

Top 6 shape functions sorted by risk contribution for a Low Risk Patient ($p = 0.0873$)



- Post menopausal.
- Moderate risk is driven by treatable cancers (endometrial carcinoma, non-invasive breast lesion) and a benign abdominal tumor — conditions that respond well to outpatient treatment.
- Notably, inpatient and ER visit counts are low, suggesting the patient is being managed effectively without repeated hospitalization.

Modularity of GAMs ---

Model Decomposition

$$\text{prediction} = \underbrace{\beta_0}_{\text{bias}} + \underbrace{\sum_j f_j(x_j)}_{\text{individual features}} + \underbrace{\sum_{i \neq j} f_{ij}(x_i, x_j)}_{\text{pairwise interactions}}$$

This structure helps us **clearly understand the model**: for each patient, we can compute which term is resulting in what risk score and rank terms based on their contribution.

Sorting Terms by Importance ---

- For each patient, we can compute which term contributes what risk score
- Terms are ranked by their individual risk score contribution for that patient
- This ranking identifies which features are driving the risk prediction for each specific patient

Although the readmission model has over 4,000 terms, in practice only a small number are relevant per patient — **making even large models locally interpretable**.

Feature Shaping vs. Expert Discretization ---

Expert Discretization

Experts manually convert continuous features into binary ranges (e.g., age 18–39, 40–54, ...). Used in the original logistic regression model.

GAM Feature Shaping

GAMs **learn** the function shape directly from data — no manual binning required. GAM with continuous features outperformed expert-discretized LR by $\sim$0.01 AUC.

Conclusion

Expert discretization introduces unnecessary rigidity — GAMs can discover finer structure (e.g., a jump in pneumonia risk at age 67) that experts would not define by hand.

Correlation $\neq$ Causation ---

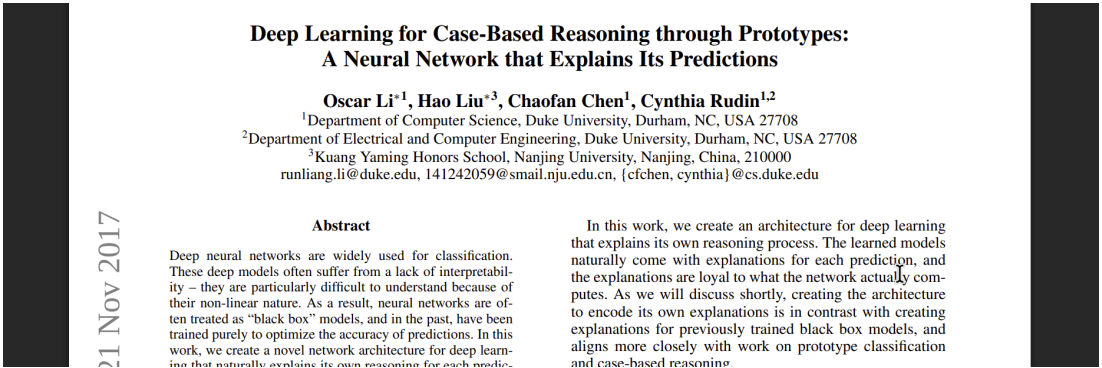
⚠ Important Warning

GAMs and GA^{2}Ms are intelligible — **but they are not causal!**

- What we see in plots are **associations** captured from data, not causal implications
- It is often easy to confuse intelligibility of predictive models with causality

Please don't make that mistake!

Deep Learning for Case-Based Reasoning through Prototypes ---



Deep Learning for Case-Based Reasoning through Prototypes: A Neural Network that Explains Its Predictions

Oscar Li[*1], Hao Liu[*3], Chaofan Chen[1], Cynthia Rudin[1,2]
[1]Department of Computer Science, Duke University, Durham, NC, USA 27708
[2]Department of Electrical and Computer Engineering, Duke University, Durham, NC, USA 27708
[3]Kuang Yaming Honors School, Nanjing University, Nanjing, China, 210000
runliang.li@duke.edu, 141242059@smail.nju.edu.cn, {cfchen, cynthia}@cs.duke.edu

Abstract

Deep neural networks are widely used for classification. These deep models often suffer from a lack of interpretability – they are particularly difficult to understand because of their non-linear nature. As a result, neural networks are often treated as "black box" models, and in the past, have been trained purely to optimize the accuracy of predictions. In this work, we create a novel network architecture for deep learning that naturally explains its own reasoning for each predic-

In this work, we create an architecture for deep learning that explains its own reasoning process. The learned models naturally come with explanations for each prediction, and the explanations are loyal to what the network actually computes. As we will discuss shortly, creating the architecture to encode its own explanations is in contrast with creating explanations for previously trained black box models, and aligns more closely with work on prototype classification and case-based reasoning.

Prototype-Based Approaches

(Li et al. 2017)

Contributions (Prototypes) ---

Novel Architecture

Proposed and developed a novel network architecture for deep learning that:

- Explains its own reasoning for each prediction
- **Not** post-hoc explanations
- Prototypes are learned **during training**
- Explanations are **faithful** to what the network computes

Motivation (Prototypes) ---

- ML models are increasingly deployed to answer societal questions $\Rightarrow$ **interpretability/transparency**
- **Radiology:** lack of transparency poses challenges to FDA approval for deep learning models
- Neural nets are particularly difficult to understand because of the high degree of non-linearity

Related Work: Post-hoc Explanations ---

Problem with Post-hoc Interpretability

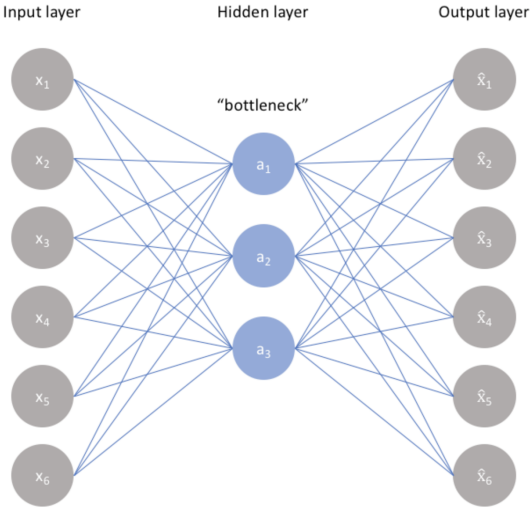
Neural networks are trained purely for accuracy; explanations are generated **after the fact** by a separate model — not by the network itself.

- Post-hoc explanations may not reflect what the network **actually computes**
- **Multiple conflicting yet convincing explanations** can be generated for the **same prediction**.
- Explanations **inherit the assumptions** of the explanation model, not the original network

This Paper's Proposal

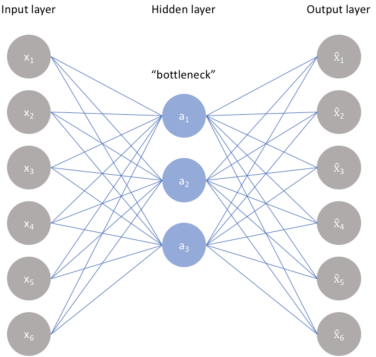
Build interpretability **into** the architecture — so explanations are loyal to what the network computes, with no separate modeling effort required.

Background: Autoencoder (1/4) ---



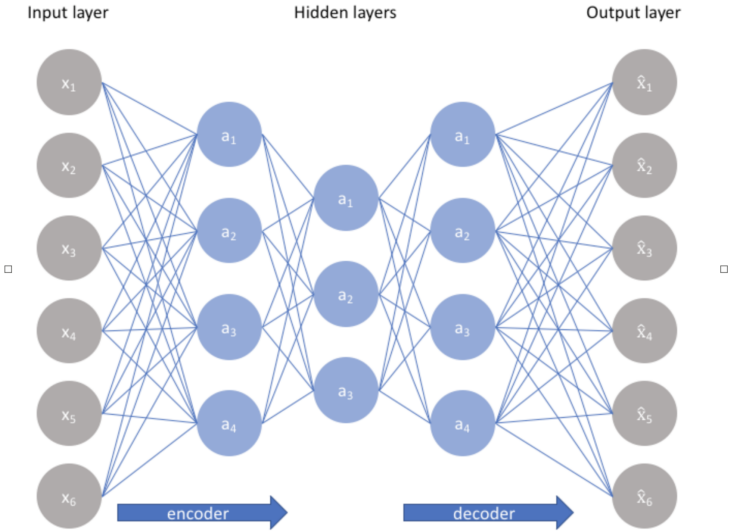
Non-linear Dimensionality Reduction and Reconstruction

Background: Autoencoder (2/4) ---



An autoencoder is a neural network trained to compress an input into a low-dimensional latent representation and then reconstruct the original input from it.

Background: Autoencoder (3/4) ---



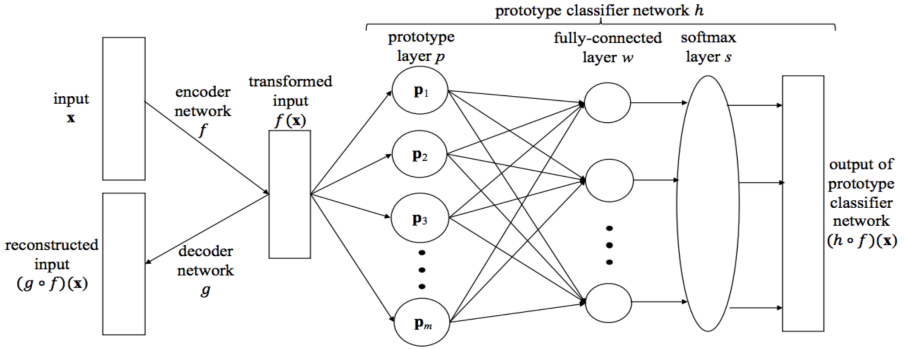
Background: Constructing an Autoencoder ---

- Constrain the number of nodes present in the hidden layer(s) of the network
  - Limiting the amount of information that can flow through the network
- By penalizing the network according to the **reconstruction error**, the model can learn the most important attributes of the input data
- By penalizing reconstruction error, the model is forced to retain only the most important attributes of the input in the latent representation

Key Insight

The encoding will learn and describe **latent attributes** of the input data.

Proposed Network Architecture ---



Autoencoding

Network Architecture: Two Components ---

1. Autoencoder

Encoder $f : \mathbb{R}^p \to \mathbb{R}^q$ compresses input $\mathbf{x}$ into a low-dimensional latent vector $f(\mathbf{x})$.

Decoder $g : \mathbb{R}^q \to \mathbb{R}^p$ reconstructs the original input $(g \circ f)(\mathbf{x}) \approx \mathbf{x}$.

The decoder also **visualizes prototypes**: given p_j in latent space, $g(p_j)$ produces a human-readable image.

2. Prototype Classifier h

Prototype layer p: computes squared distances $\|f(\mathbf{x}) - p_j\|^2$ to each of m learned prototypes.

Fully-connected layer w: combines distances into per-class scores $Wp(f(\mathbf{x}))$.

Softmax layer s: converts scores into a probability distribution over K classes.

The final prediction $(h \circ f)(\mathbf{x})$ is naturally explained: *"classified as class k because it resembles prototype p_j."*

Cost Function ---

The full cost function of the proposed architecture is:

$$L((f, g, h), D) = E(h \circ f, D) + \lambda R(g \circ f, D) + \lambda_1 R_1 + \lambda_2 R_2$$

Term	Role
D	Training dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$
f	Encoder network: $\mathbb{R}^p \to \mathbb{R}^q$
g	Decoder network: $\mathbb{R}^q \to \mathbb{R}^p$
h	Prototype classifier network: $\mathbb{R}^q \to \mathbb{R}^K$
E	Classification accuracy (cross-entropy)
R	Autoencoder reconstruction fidelity
R_1	Prototypes close to real training examples — **realism**
R_2	Training examples close to some prototype — **coverage**

$\lambda, \lambda_1, \lambda_2$ balance accuracy vs. interpretability. In the paper, all three are set to 0.05.

Prototypes: Definition and Learning ---

A prototype $p_j \in \mathbb{R}^q$ is a vector in the latent space learned during training — decoded via $g(p_j)$ to produce a human-readable image.

Prototypes are learned like any other network parameter: by **backpropagation**, minimizing the joint cost function L. Two regularization terms guide their placement:

R_1 — Realism

$$\frac{1}{m}\sum_{j=1}^{m}\min_i \|p_j - f(x_i)\|^2$$

Each prototype must be close to at least one training example → decoded images look realistic.

R_2 — Coverage

$$\frac{1}{n}\sum_{i=1}^{n}\min_j \|f(x_i) - p_j\|^2$$

Each training example must be close to at least one prototype → prototypes cover the full input space.

Where n is the number of training examples and m is the number of prototypes (a hyperparameter; not necessarily equal to the number of classes K). In the paper, $m = 15$ for MNIST ($K = 10$).

Advantages of the Proposed Architecture ---

Key Advantages

- **Automatically learns** useful features (non-linear dimensional reduction)
- Suitable for **high-dimensional data** such as images
- Prototype vectors can be **decoded and visualized** (same latent space as encoded inputs)
- Ability to interpret **without post-hoc analysis**

Results: MNIST Data ---

Case Study 1: MNIST ---

Test accuracy: 99.22%

on par with a standard CNN (99.23%); no accuracy sacrificed for interpretability

Original vs. Reconstructed

Original 

Autoencoder Reconstructed 

Reconstruction error: 4.22 (avg. ≈ 0.005 per pixel) — decoder faithfully maps latent vectors to pixel space, ensuring decoded prototypes look realistic

Learned Prototypes

Multiple prototypes per class (e.g., three "6"'s) capture different writing styles — enabled by R_1 (realism) and R_2 (coverage).

Learned Weight Matrix ---

	0	1	2	3	4	5	6	7	8	9
8	-0.07	7.77	1.81	0.66	4.01	2.08	3.11	4.10	-20.45	-2.34
9	2.84	3.29	1.16	1.80	-1.05	4.36	4.40	-0.71	0.97	-18.10
0	-25.66	4.32	-0.23	6.16	1.60	0.94	1.82	1.56	3.98	-1.77
7	-1.22	1.64	3.64	4.04	0.82	0.16	2.44	-22.36	4.04	1.78
3	2.72	-0.27	-0.49	-12.00	2.25	-3.14	2.49	3.96	5.72	-1.62
6	-5.52	1.42	2.36	1.48	0.16	0.43	-11.12	2.41	1.43	1.25
3	4.77	2.02	2.21	-13.64	3.52	-1.32	3.01	0.18	-0.56	-1.49
1	0.52	-24.16	2.15	2.63	-0.09	2.25	0.71	0.59	3.06	2.00
6	0.56	-1.28	1.83	-0.53	-0.98	-0.97	-10.56	4.27	1.35	4.04
6	-0.18	1.68	0.88	2.60	-0.11	-3.29	-11.20	2.76	0.52	0.75
5	5.98	0.64	4.77	-1.43	3.13	-17.53	1.17	1.08	-2.27	0.78
2	1.53	-5.63	-8.78	0.10	1.56	3.08	0.43	-0.36	1.69	3.49
2	1.71	1.49	-13.31	-0.69	-0.38	4.55	1.72	1.59	3.18	2.19
4	5.06	-0.03	0.96	4.35	-21.75	4.25	1.42	-1.27	1.64	0.78
2	-1.31	-0.62	-2.69	0.96	2.36	2.83	2.76	-4.82	-4.14	4.95

- Each row is a prototype p_j (visualized on the left); each column is a digit class. The most negative weight (shaded) indicates the **visual class** of the prototype.
- The last prototype (visual class "2' ') is an exception — it has strong negative weights for classes 7 and 8.

Ablation Study on Cars Data ---

Components are removed one by one to measure their individual contribution.

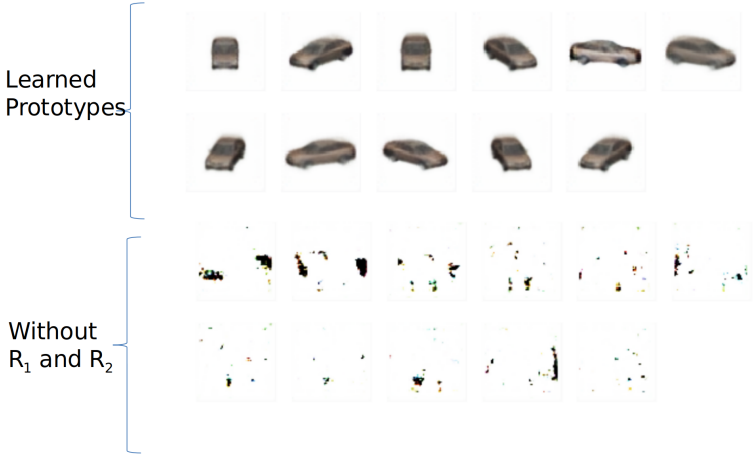
Model	Test Accuracy
Full model (autoencoder + prototype layer)	99.22%
Prototype layer replaced by fully-connected layer	99.24%
No prototype layer, no decoder (standard CNN)	99.23%

All three variants achieve comparable accuracy — adding the autoencoder and prototype layer does **not** hurt predictive performance.

Key Takeaway

Interpretability comes at virtually **no accuracy cost** — the extra terms in L simply steer the model toward a more interpretable solution among equally accurate ones.

Ablation Study on Cars Data (1/2) ---



- **With R_1 and R_2:** prototypes are clear, recognizable car images
- **Without R_1 and R_2:** prototypes are noisy and uninterpretable

Ablation Study on Cars Data (2/2) ---

- **Without R_1:** prototypes are noisy and unrecognizable
- **Without R_2:** prototypes are redundant and lack diversity

Thank You!

References --- I

Caruana, Rich, Yin Lou, Johannes Gehrke, Paul Koch, Marc Sturm, and Noemie Elhadad. 2015. "Intelligible Models for Healthcare: Predicting Pneumonia Risk and Hospital 30-Day Readmission." In *Proceedings of the 21th Acm Sigkdd International Conference on Knowledge Discovery and Data Mining*, 1721–30.

Li, Oscar, Hao Liu, Chaofan Chen, and Cynthia Rudin. 2017. "Deep Learning for Case-Based Reasoning Through Prototypes: A Neural Network That Explains Its Predictions." *arXiv: 1710.04806*.

Lou, Yin, Rich Caruana, and Johannes Gehrke. 2012. "Intelligible Models for Classification and Regression." In *Proceedings of the 18th Acm Sigkdd International Conference on Knowledge Discovery and Data Mining*, 150–58.